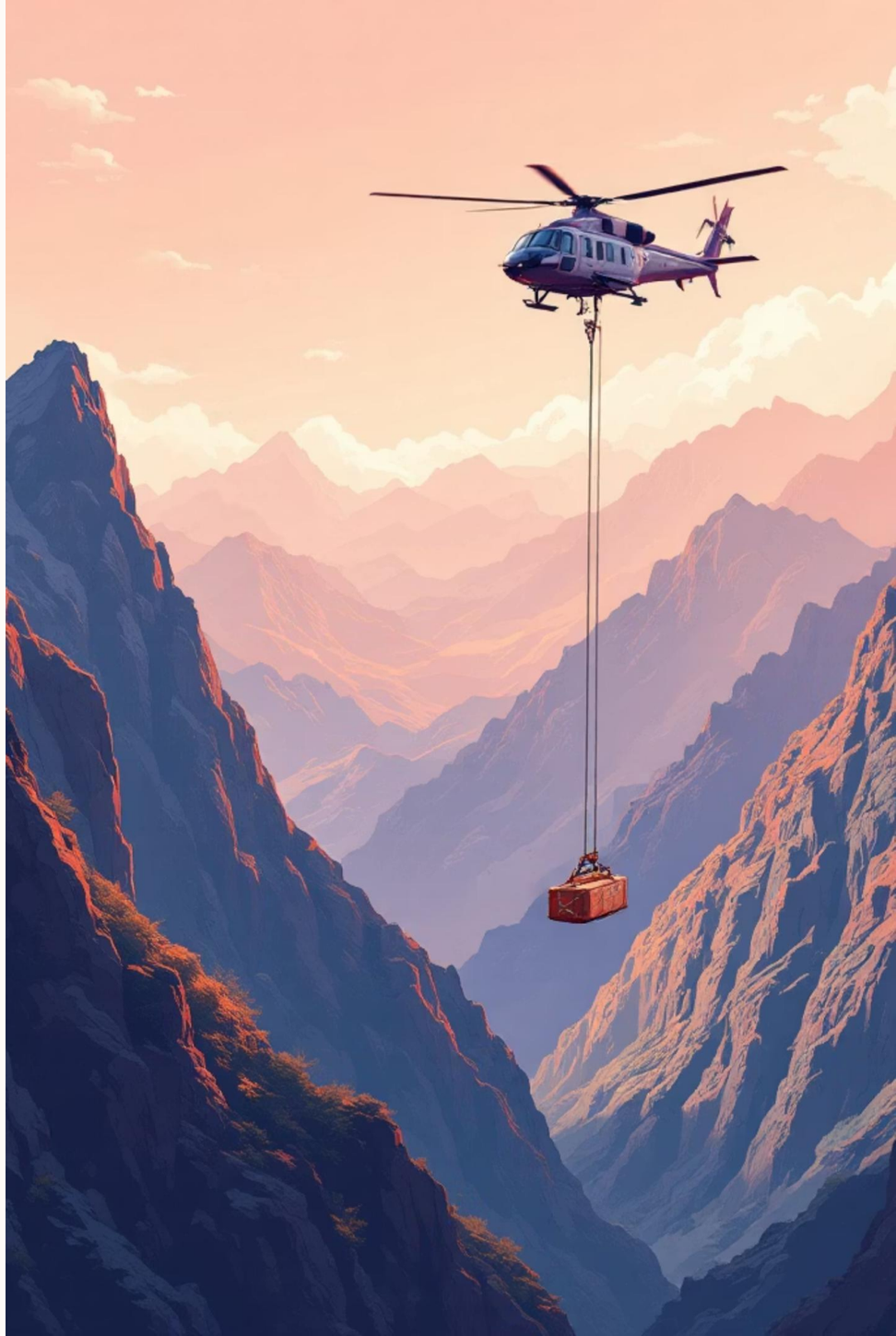
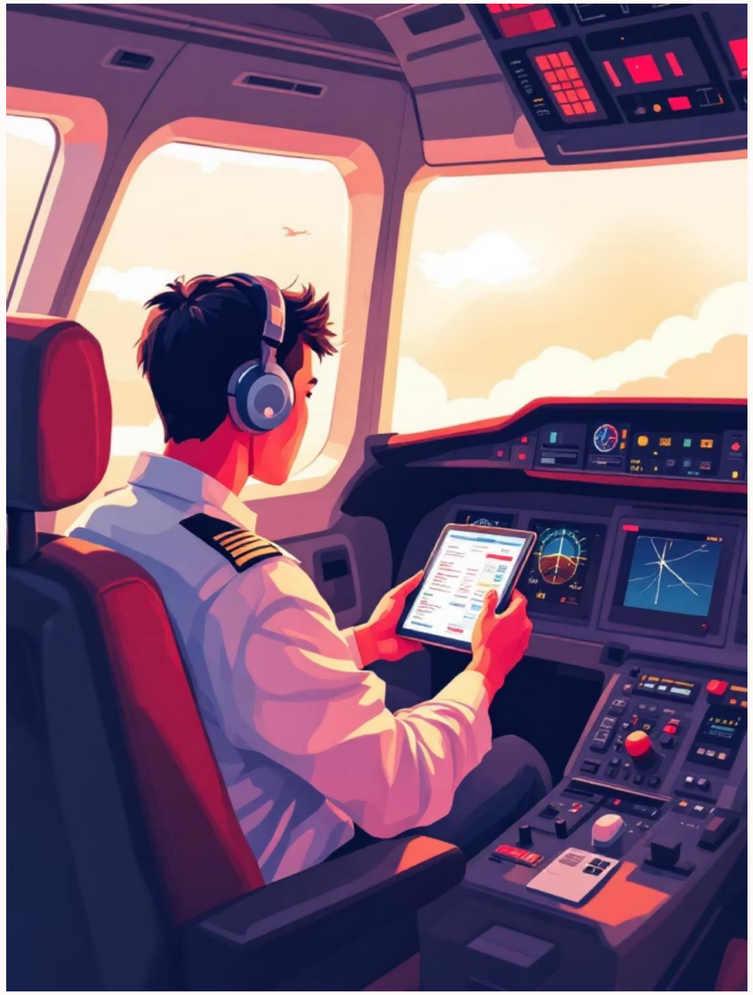
«Воздушный извозчик» — Система учёта грузовых вертолётов

Лабораторная работа №4 · Вариант 3 · Тюшко Вадим · Группа 12



Актуальность и постановка задачи

- Проблема: централизованный учёт вертолётов, экипажей и рейсов в горной зоне.
- Контроль: налёт, ресурс техники, расчёт заработка экипажа, формирование отчётов.
- Цель: надёжная мобильная/CLI-система для оперативного учёта и контроля рейсов.



Функциональные требования (Use Case)

- Роли: Командир авиаотряда (полный доступ), Член экипажа (только свои данные).
- Операции: CRUD для вертолётов/экипажей/рейсов, отчёты, фильтры по периоду.
- Контроль ресурса, вычисление заработка, генерация статистики и экспорт отчётов.

Технологический стек

C (C99)

Язык реализации: строгий, эффективный, низкоуровневый контроль памяти.

SQLite

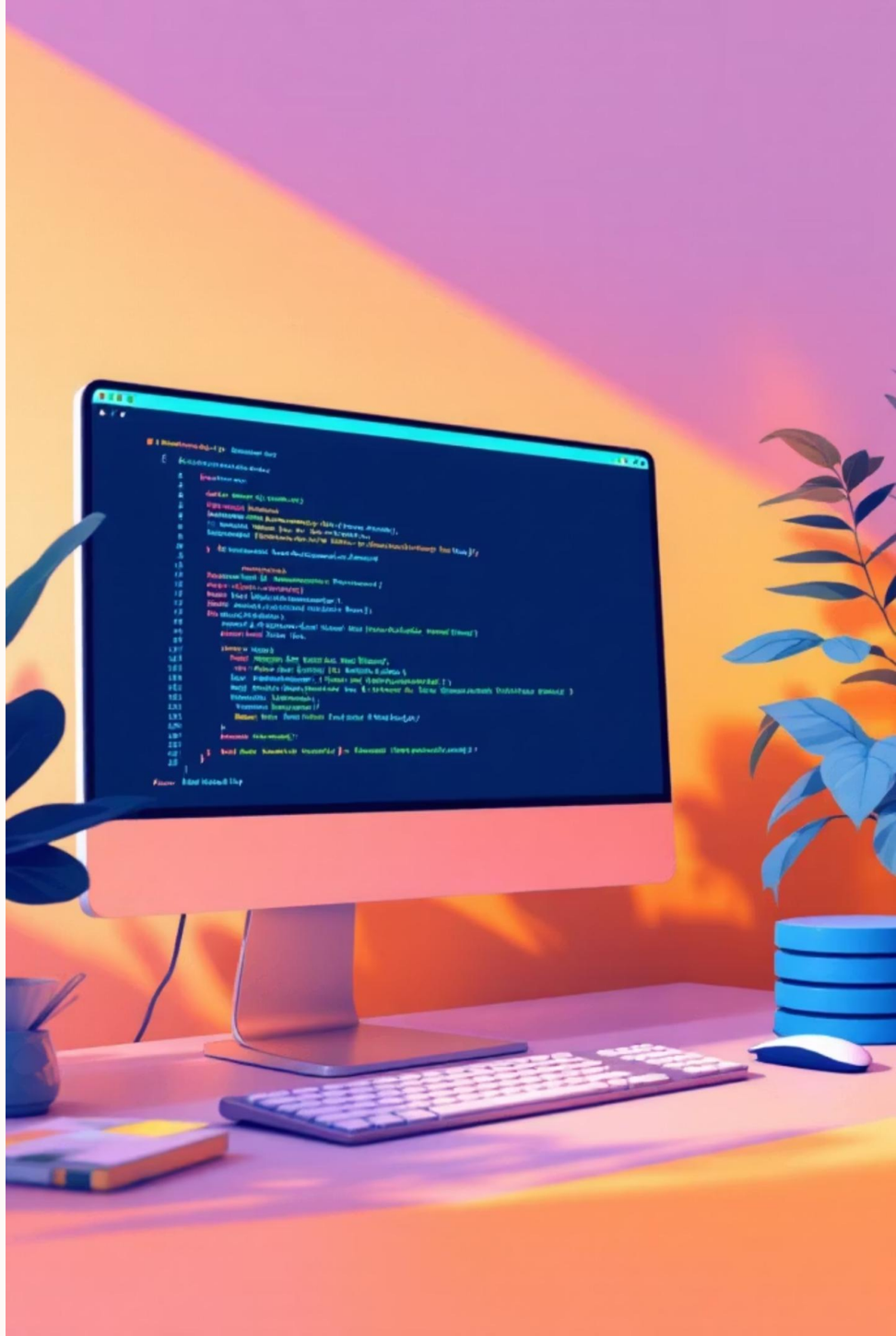
Встроенная СУБД: простая деплойка, ACID, файлы .db для тестов и CI.

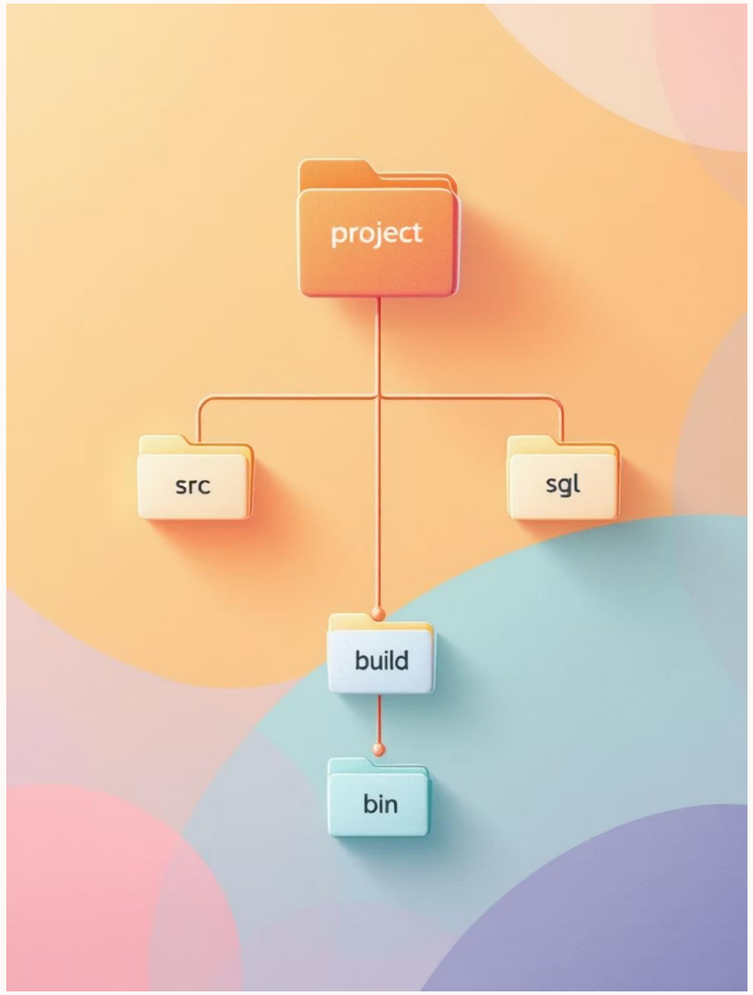
CI/CD — GitHub Actions

Автоматическая сборка, создание БД, запуск тестов (6 проверок).

Документация

Wiki (Markdown), UML (draw.io / PlantUML) — спецификации и глоссарий.





Структура проекта (модель КИС)

- Interface: includes/database.h — API доступа к БД.
- Kernel: src/db_queries.c, auth.c, salary.c, image_utils.c — бизнес-логика.
- Client: src/main.c — интерфейс командной строки / точка входа.
- Папки: bin, build, includes, src, sql, images — разделение обязанностей.

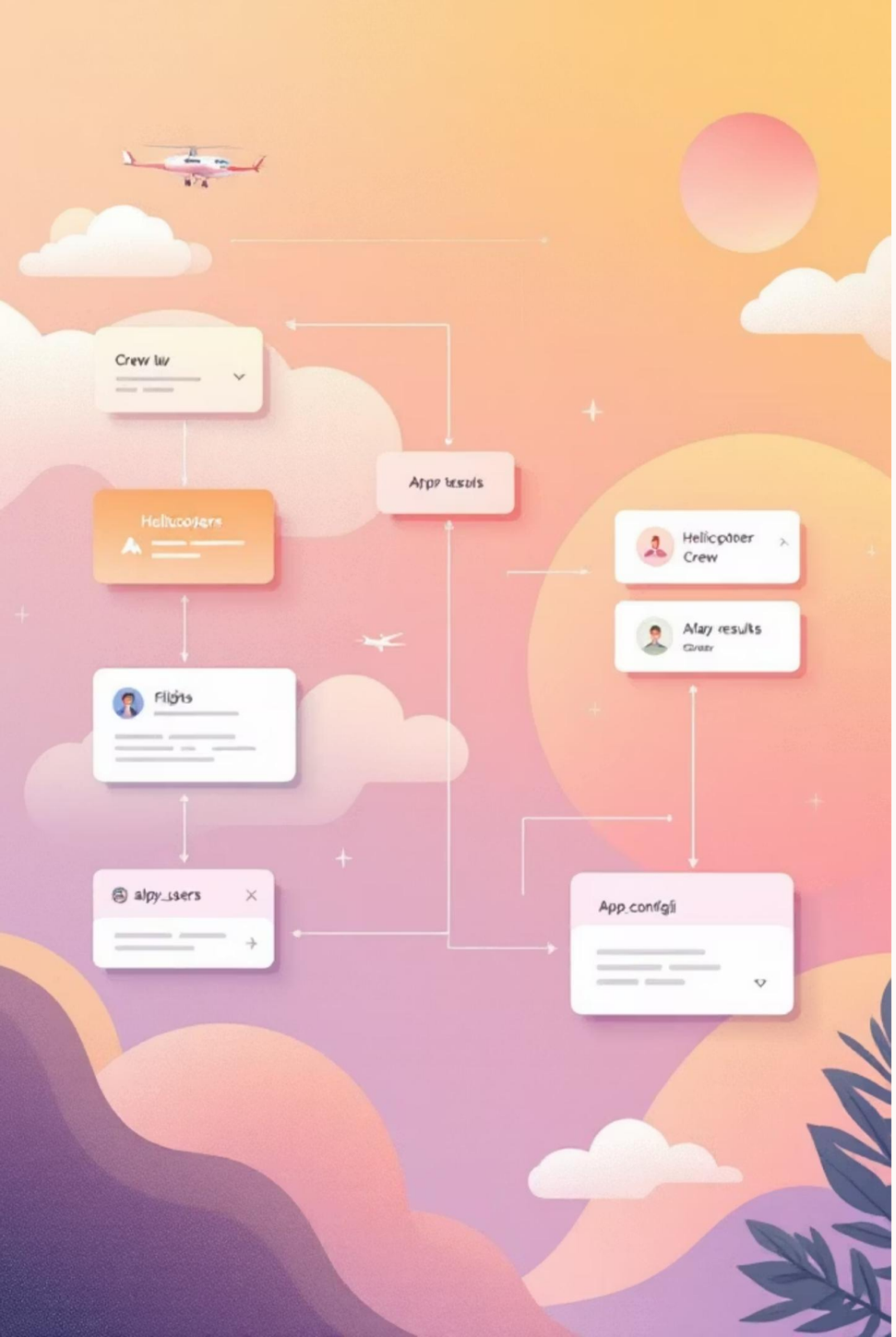
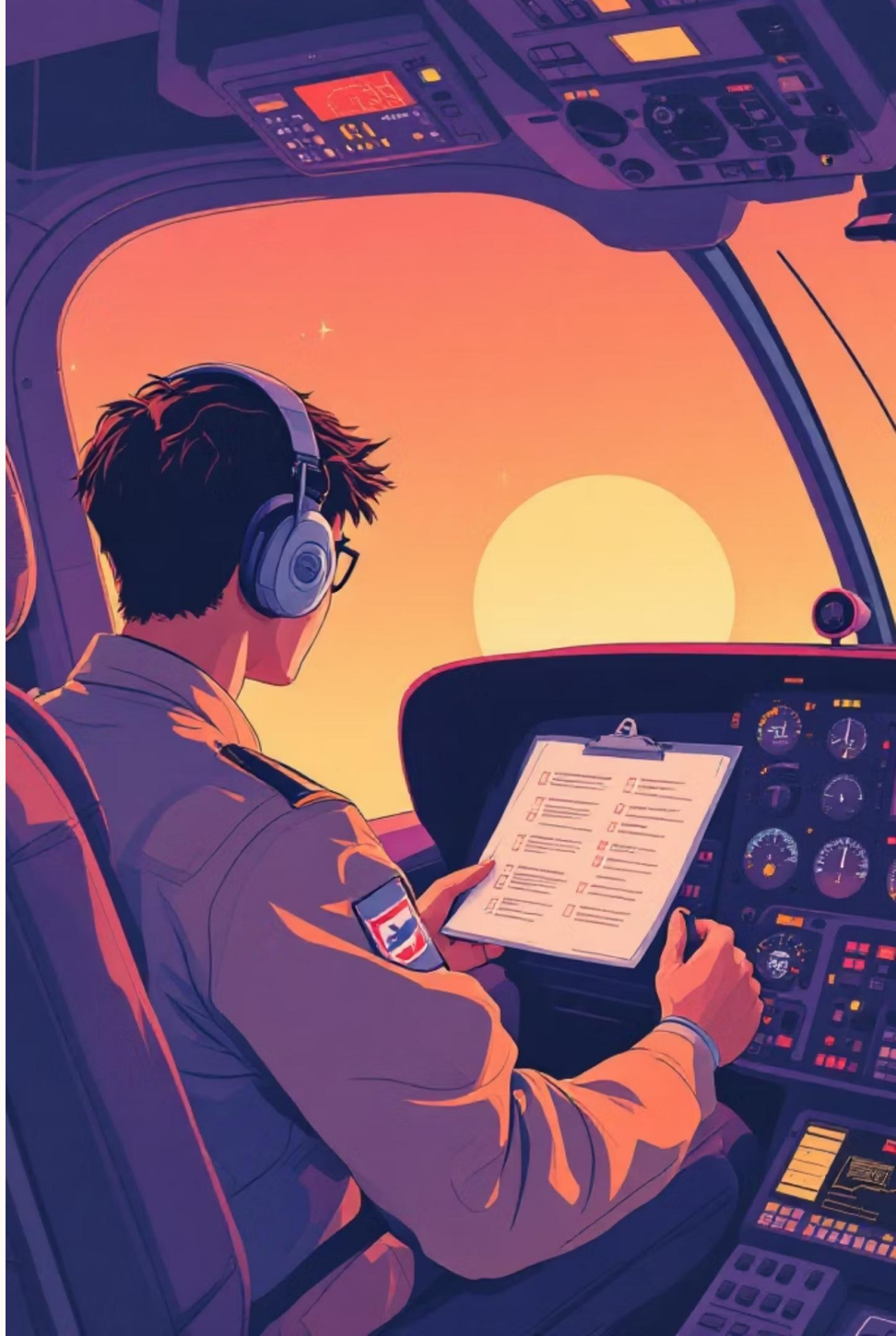


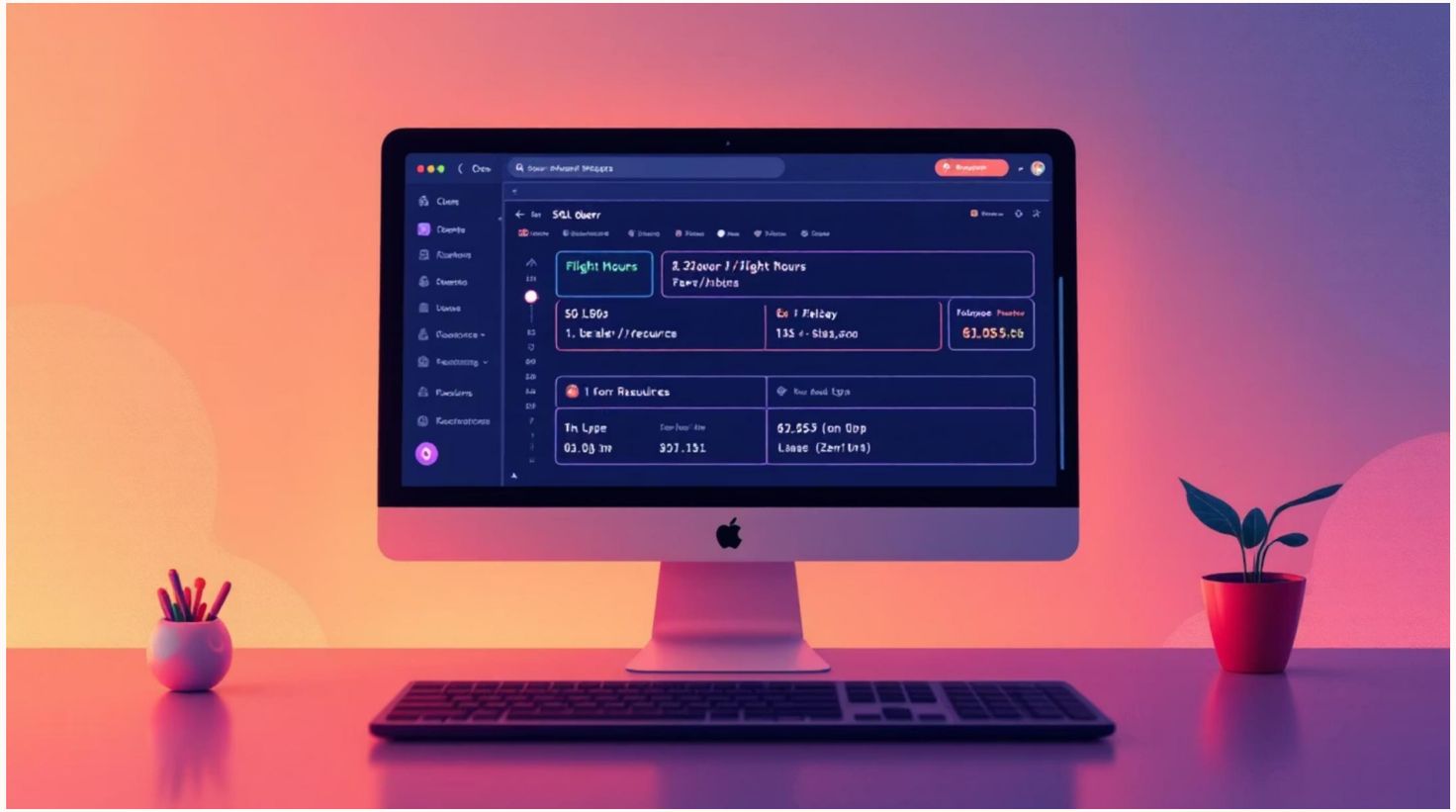
Схема базы данных

Таблицы: helicopters, crew, flights, app_users, salary_results, app_config. Реализованы внешние ключи и ограничения CHECK. Для фото — поле photo_path.

Реализация ключевого функционала

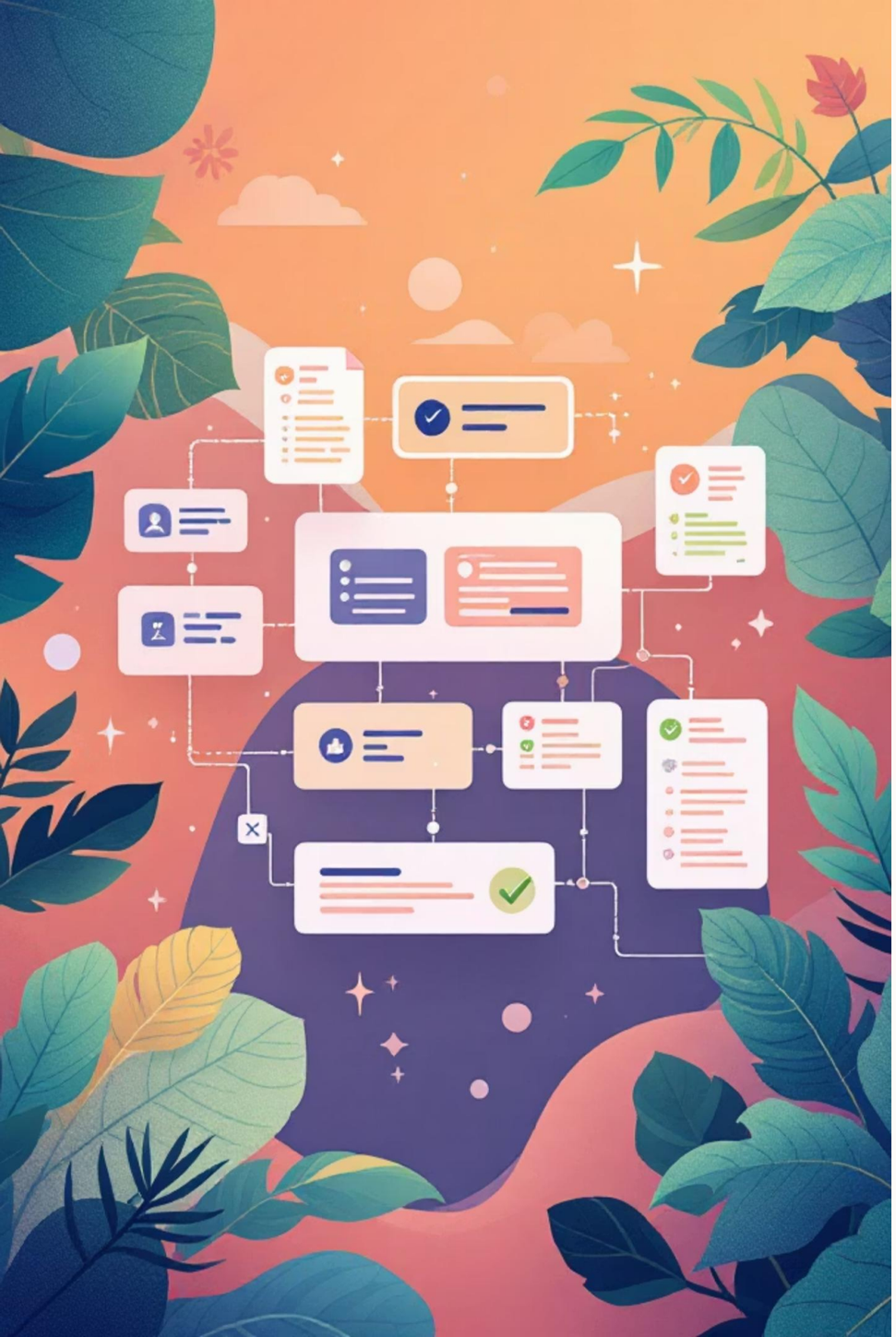
- Проверка ресурса вертолёта: до добавления рейса выполняется запрос остатка ресурса — запись отклоняется при превышении.
- Расчёт заработка экипажа: гибкая логика — ставка 5% для обычных рейсов, 10% для спецрейсов (реализовано в salary.c).
- Kanban: 16 задач — 1–8 назначены студенту, 9–16 — второму участнику; трекинг статусов в Wiki.





Примеры запросов и сценарии

- Вывод налёта и остатка ресурса по вертолёту за период.
- Список рейсов вертолёта за указанный интервал с фильтром спец/обычных.
- Поиск вертолёта с максимальным числом рейсов.
- Обработка ошибок: попытка добавить рейс при недостаточном ресурсе — отклонение и лог.



Тестирование и CI/CD

- Локально: сборка через Makefile, SQL-скрипты для инициализации и тестовых данных.
- GitHub Actions: workflow создаёт БД, запускает 6 автоматических проверок (сборка, unit-тесты, интеграционные SQL-тесты, статический анализ).
- Доказательство: скриншоты успешного прохождения CI и логи тестов в репозитории.

Документация, UML и выводы

Документация

Wiki репозитория fpmi-tpmp2026/tpmp-gr12a-lab4-n-zar: требования, глоссарий, инструкции.

UML

Диаграммы: прецедентов, классов, последовательностей, компонентов и развёртывания.

Итоги

Проект реализован полностью: все 16 задач выполнены, приложение готово к эксплуатации, CI настроен. Навыки: C + SQLite, Git, GitHub Actions, UML.